



# TREES

Dr. Priyanka N

Assistant Professor Senior Grade I

School of Computer Science & Engineering  
VIT,Vellore.

|                                                                                                                                                                                                                                                                                                                                                                                                                                   |                               |                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|----------------|
| <b>Module:1</b>                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Algorithm Analysis</b>     | <b>8 hours</b> |
| Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method. |                               |                |
| <b>Module:2</b>                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Linear Data Structures</b> | <b>7 hours</b> |
| Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.                                                                         |                               |                |
| <b>Module:3</b>                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Searching and Sorting</b>  | <b>7 hours</b> |
| Searching: Linear Search and binary search – Applications.<br>Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.                                                                                                                                                                                                                                       |                               |                |
| <b>Module:4</b>                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Trees</b>                  | <b>6 hours</b> |
| Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the $k^{\text{th}}$ minimum element.                                                                                                                                                                                                       |                               |                |
| <b>Module:5</b>                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Graphs</b>                 | <b>6 hours</b> |
| Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.                                                                                                                                                                                                                     |                               |                |
| <b>Module:6</b>                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Hashing</b>                | <b>4 hours</b> |
| Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.                                                                                                                                                                                                                                                          |                               |                |
| <b>Module:7</b>                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Heaps and AVL Trees</b>    | <b>5 hours</b> |
| Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).                                                                                                                                                                                                                                                                                         |                               |                |

# TREES

- Introduction to Trees & Terminologies
- Binary Tree – Definition and Properties
- Binary Tree Representation
- Tree Traversals
- **Expression Trees**
- Binary Search Trees
- Operations on Binary Search Trees
  - Insertion
  - Deletion
  - Finding Max and Minimum
  - Finding the  $k^{\text{th}}$  Minimum Element

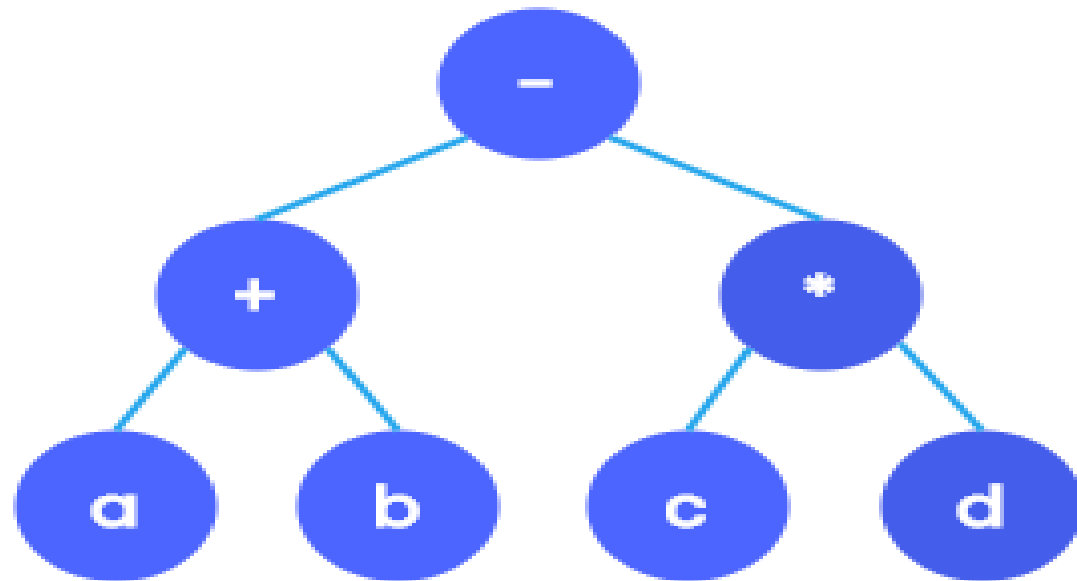
# Expression Trees

- An expression tree is a tree built up from the operands as the leaves of binary tree and operators as the non -leaves of binary tree.
- It is a special kind of binary tree in which
  - Each leaf node contains single operand
  - Each non-leaf node contains a single binary operator
  - The left and right subtrees of an operator node represent sub-expression that must be evaluated before applying the operator at the root of the subtree.

# Idea

- Levels in a binary expression tree represent the precedence of operators.
- Operators at the lower level must be evaluated first
- Then the operators at the next level and so on and at the last operator at the root node is applied and there by the expression is evaluated.

Ex :  $X = (a + b) - (c * d)$



# Constructing an Expression Tree

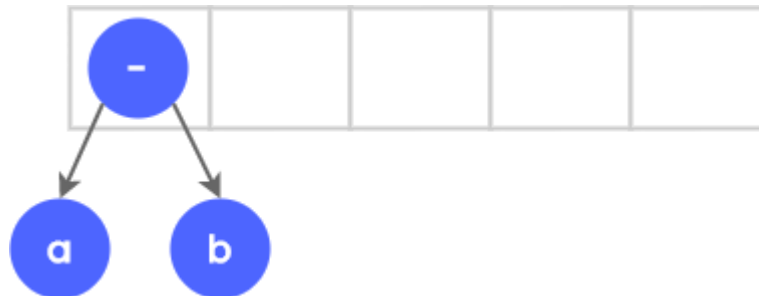
- Convert the given Expression into Postfix form.
- Repeat the following steps:
  - If the **character is an operand**, create a node and **PUSH** it on to the stack.
  - If the **character is an operator**, **POP** two values from the stack and make a node keeping the operator as root node and the two items as the left and the right child.
  - Push the newly generated node into the stack.

Consider the Post fix Expression: **a b - c \***

- Step 1: First two characters are operands, PUSH it onto the stack.



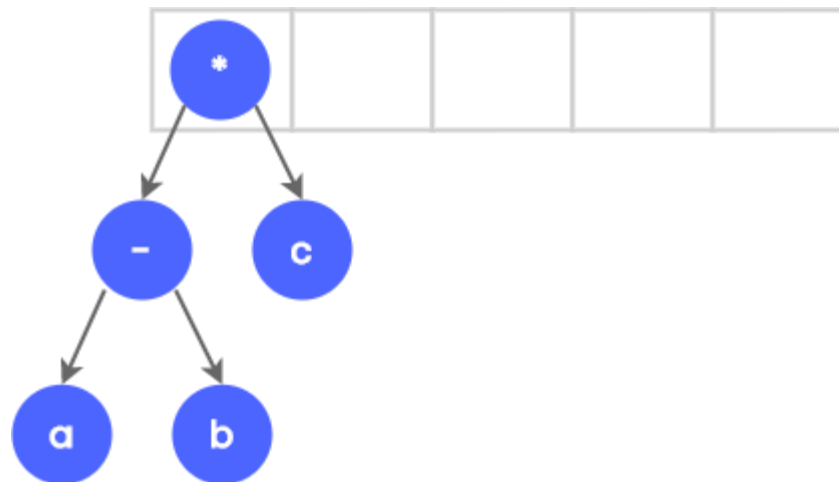
- Step 2: Next, “-” Symbol is read, “a” and “b” are popped from the stack and is added as the child of node “-” . A pointer to the new node is now pushed onto the stack.



- Step 3: Next, “c” is read and is pushed onto the stack.



- Step 4: Next, “\*”, so pop “c” and “-” from the stack and add it as child of “\*”



# Construction of Expression Tree

```
ConstructExpressionTree(postfix[])
```

```
{
```

```
    // Traverse through every character of input expression
```

```
    for (int i=0; i<strlen(postfix); i++)
```

```
    {
```

```
        // If operand, simply push into stack
```

```
        if (!isOperator(postfix[i]))
```

```
        {
```

```
            t = newNode(postfix[i]);
```

```
            stack.push(t);
```

```
        }
```

**else // If operator**

{

t = newNode(postfix[i]);

**// Pop two top nodes**

t1 = stack.top(); **// Store top**

stack.pop(); **// Remove top**

t2 = stack.top();

stack.pop();

**// make them children to the created node**

t->right = t1;

t->left = t2;

**// Add this subexpression to stack**

stack.push(t);

}

**}// End of For Loop**

**// only element will be root of expression tree**

t = stack.top();

stack.pop();

**return t;**

}